Article

Attacker Cognitive Loop: An Interpretable Rule-Based Engine for Estimating, Predicting, and Engaging Adaptive Attackers

Jiahao Lai [1,†]

[1] ASSCOR Project; chins-xing@proton.me

Abstract

Attackers and defenders operate under fundamentally unequal information endowments: attackers choose when, where, and how to probe; defenders react to signals that arrive too late and too sparse. This paper presents the Attacker Cognitive Loop (ACL), a system designed to actively reverse this information asymmetry — not by blocking attacker actions, but by guiding them through lightweight decoys that reveal intent earlier than the attacker intends. ACL closes the loop between observation and intervention, turning the defender's information disadvantage into a controllable orchestration problem. The engine is implemented as an interpretable, rule-based extension of the open-source ASSCOR platform, with every component mapped to a public repository path and branch. Its heuristic weights are explicitly not optimal; they are provided solely to demonstrate closed-loop mechanics and to serve as a baseline for community-driven replacement. We formulate the research question — *how AI, shared tooling, and individual experience jointly shape the attacker's action distribution, and whether defenders can actively reshape it through observation, deception, and guidance* — and state ten open research questions (Q1–Q10) with evidence ratings. We provide the complete formal specification of the scoring, normalization, sharpness, and utility functions, a reproducible four-round closed-loop trace with exact numerical values, a sensitivity analysis of the hand-tuned weights, and simulated per-round latency benchmarks.

Keywords: active defense; cyber deception; attacker modeling; honeypot; decision sharpness; moving target defense; I-POMDP; interpretable AI; open research questions; reproducible research

---

1. Introduction

Classical defense embeds blocking controls: access control and isolation make undesirable states unreachable. Surveys of moving-target defense (MTD) have documented two inherent weaknesses of this paradigm. First, a *cognitive limitation*: defenders cannot enumerate all latent attacker paths, and attackers enjoy an information advantage during prolonged reconnaissance. Second, a *temporal asymmetry*: attackers can plan over long horizons while defenders must respond in real time, and patch lag leaves a window of opportunity.

At the heart of the two weaknesses above lies a single structural fact: attackers hold an inherent information advantage. They observe the defender's environment at their chosen pace, select which paths to probe, and can wait for the defender's response patterns to emerge. Defenders, by contrast, see only the artifacts of actions already taken — alerts, logs, and blocked connections. This is not a technical failure of firewalls or IPS; it is a cognitive asymmetry built into the defender's observational posture.

Guided active defense, as we define it, is an attempt to reverse this asymmetry. Instead of waiting for the attacker to reveal intent through damage, the defender proactively deploys ephemeral decoys that appear to reduce the attacker's uncertainty — inducing the attacker to verify hypotheses early, thereby exposing TTPs and intent before the attack reaches its objective. ACL is designed as an

orchestration engine for exploring this reversal: it estimates what the attacker believes, predicts what the attacker will do next, and selects decoys that maximize information gain while minimizing the defender's exposure. The current implementation provides a fully auditable rule-based baseline against which more principled models can be compared.

Guided active defense does not aim to block everything; it aims to create an *information asymmetry* in the defender's favor. By deploying lightweight decoy targets and fabricated intelligence, the defender induces the attacker to abandon slow, patient operation in favor of rapid verification — exposing tactics, techniques, and procedures (TTPs) and intent earlier than they would otherwise be revealed. Intelligence collected from each interaction feeds the next, more accurate round of deception, forming a positive feedback loop: *guide* → *collect* → *analyze* → *decide* → *guide again*.

In the MITRE Engage taxonomy, engagement is used here as a guidance tool, not a blocking tool; host isolation is demoted to a fallback when guidance fails or confirms high risk.

This work makes the following contributions:

1. We formulate the attacker-cognitive-loop research question that connects attacker modeling with adaptive deception.
2. We present an interpretable, rule-based engine (ACL) that realizes the loop: state estimation, action-distribution prediction, utility-driven engagement selection, and sharpness-driven strategy switching, with the complete formal specification of every score, normalization, sharpness, and utility term (Section 4.3.1, Section 4.5.1, Section 4.4.1).
3. We provide a reproducible four-round closed-loop trace with exact numerical values computed from the reference implementation's weights (Section 9), so the engine's behavior can be independently verified and replayed.
4. We anchor the work in a real, public codebase: the engine is an implemented extension of the open-source ASSCOR assessment platform, with every component mapped to a repository path and branch (Section 7, Table 4), so the paper is directly auditable rather than a paper-only design.
5. We state ten open research questions (Q1–Q10) about attacker behavior in the AI era, each with an evidence rating, and a validation plan.
6. We are transparent about what is implemented (the engine) versus what remains to be answered (the questions), and we analyze the sensitivity of the hand-tuned weights (Section 12.1). No assumption is presented as a conclusion.

2. Related Work

Decision-theoretic planning. Shinde and Doshi's I-POMDP^2 [1] extends partially observable Markov decision processes to model the defender's beliefs about the attacker's beliefs, capabilities, and preferences, with finite nested recursive reasoning. Their experiments observe level-3 defenders who first mislead the attacker into believing the defense is passive, then deploy decoys — a decision-theoretic justification for guidance over blocking. A full I-POMDP^2 solver is computationally prohibitive; our engine is a transparent rule-based approximation that can be replaced by richer models without changing the loop architecture. A complementary line of work learns the attacker's reward structure rather than assuming it: Shinde and Doshi [12] model behavioral preferences of cyber adversaries from system-level audit logs using inverse reinforcement learning, recovering the objective function an attacker implicitly optimizes. This is directly relevant to our *attacker state* abstraction (Section 3.1): where our engine represents the attacker's *Objective* field as an open string, an IRL-derived reward can populate it from data, and the same loop interface (Section 6.3) accepts such a model as a drop-in replacement of the rule-based state engine.

Game-theoretic deception. Zhu's tutorial [2] and Anwar & Kamhoua's attack-graph games [5] formalize signaling games, dynamic games, mechanism design, and hypergames for deception. General-sum stochastic games make pure-strategy Nash equilibrium computation PSPACE-hard, and equilibrium concepts assume symmetric optimality that poorly models the attacker's *bounded rationality*. We therefore prefer the decision-theoretic view.

AI-empowered attackers. The rapid adoption of large language models (LLMs) as autonomous offensive agents is reshaping the attacker model our hypotheses target. Recent surveys [13] document LLM-based agents that plan, execute, and chain attack steps end-to-end, producing attack sequences that are simultaneously more accessible (lowering the skill barrier, directly relevant to H6/Q6) and more homogeneous (agents trained on overlapping corpora converge on common solution patterns). This second property is exactly the mechanism behind our core question Q7 (*does AI lead to attacker strategy convergence?*, Section 10): if LLM-generated attack plans cluster around a shared strategy distribution, defenders can exploit the resulting predictability. Our validation plan (Section 11.1) makes this measurable by comparing automated, LLM- or tool-driven attack profiles (runs E1–E9) against human red-team behavior, using TTP entropy, sequence distance, and strategy-cluster analysis as the quantitative axes.

Attacker modeling and attribution. The Diamond Model of intrusion analysis [6] structures every intrusion event as four vertices — *adversary*, *capability*, *infrastructure*, and *victim* — connected by six phases of an activity thread, with meta-features (timestamp, kill-chain phase, result, direction, method, resources) describing each event. Its central axiom, that each intrusion is an event in an activity thread whose evolution is governed by the adversary, is the analytical backbone we reuse: our six-layer graph (Section 4.1) is the Diamond Model's relational projection onto a temporal substrate — the Adversary layer holds the adversary vertex, the Capability layer holds capabilities and TTPs, the Network/Dependency layers hold infrastructure, and the Identity/Evidence layers hold the victim and the meta-features. Unlike the Diamond Model, which is a retrospective analytic framework, ACL consumes the same relational structure *forward* to estimate the attacker's state and predict its next action. We also note the fundamental attribution problem raised by the shared-capability questions (Q5/Q6, Section 10): observed similarity between intrusions does not imply the same adversary, since adversaries share tooling, code, and (increasingly) AI-generated strategy.

Defensive deception and honeypot limitations. Zhang and Thing [3] survey three decades of honeypots, honeytokens, and MTD across a four-layer stack (network/system/software/data) mapped to the kill chain, and document the two structural weaknesses of honeypot-only defenses that motivate our guided-engagement design: (i) *static deployment*: if honeypots and honeytokens are left with static configuration, an adversary has enough time to infer their existence, map them, and evade them; and (ii) *pivot risk*: high-interaction honeypots, which offer a real operating-system environment, can themselves be compromised and used as a pivot point to attack other systems. Two further weaknesses are well-known in practice: (iii) *low-interaction honeypots have low fidelity*, so sophisticated attackers can fingerprint and bypass them; and (iv) *passivity*: traditional honeypots wait for the attacker and yield little intelligence about intent until the attacker has already committed. Our loop addresses (i) by making engagement a *function of the predicted action distribution* that changes each round, (ii) by the *sufficiency principle* (Section 4.4.2) — decoys expose only a plausible break-in appearance with no real system to compromise — (iii) by keeping decoys lightweight and cheap enough to deploy at single-host scale, and (iv) by using decoys as *active sensors* whose interaction chains feed the state engine. Recent adaptive-deception systems pursue the same direction: Siren [14] combines deception-generation with adaptive analysis to tailor decoys to an observed adversary, and AI-driven adaptive honeypots have been proposed for dynamic deployment selection. Our contribution differs in the control loop: rather than adapting decoys heuristically, ACL selects interventions by an explicit utility function over a predicted action distribution, and — crucially — decides *whether to engage at all* through the sharpness-driven strategy (Eq. 16) and the defensive fallback, which adaptive-honeypot systems do not model.

MTD design principles. MTD surveys [4] contribute five design principles (coverage, unpredictability, timeliness, superstability, functional equivalence) that constrain any future MTD component. We adopt honeypot/honeytoken as the primary guidance mechanism and treat MTD as an optional deterministic-weakening component.

Behavior standardization. MITRE ATT&CK [7] provides the semantic layer for attack behavior. Our prediction engine maps action distributions onto TTP distributions rather than replacing ATT&CK, and our state engine infers intent from observed TTPs through an explicit technique→intent table. Table 1 lists the mapping used by the reference implementation (the full table is Section 4.2); the same table drives the reverse projection $P(TTP_i) = \sum_j P(TTP_i \mid Action_j)\, P(Action_j)$ (Section 4.3.1). This makes the ATT&CK→state-machine bridge explicit, auditable, and replaceable: unknown techniques remain `unknown` rather than being guessed.

ATT&CK technique	Inferred intent	Example
T1595, T1592, T1590, T1046	recon	Active Scanning (T1595)
T1110, T1003, T1555, T1078	credential	Brute Force (T1110)
T1021, T1570, T1550, T1028	lateral	Remote Services (T1021)
T1567, T1048, T1030, T1537	data_theft	Exfiltration Over Web Service (T1567)
T1190, T1193, T1566	web_attack	Exploit Public-Facing Application (T1190)

Table 1. TTP → intent mapping used by the state engine (Section 4.2). Techniques outside this table remain `unknown`; the mapping is open data, not engine logic.

Risk propagation. SRD (Systemic Risk Dynamics) models risk diffusion over a network graph [9]; we reuse its substrate for the topology layer and as the dynamic state-evolution layer of the cognitive loop.

Our work is built on ASSCOR (ASSess + CORE), an open-source security acceptability assessment platform. ASSCOR provides the three substrates that ACL requires: (i) a six-layer temporal graph (Network, Dependency, Identity, Attacker, Capability, Evidence) that carries observations and alerts as temporal events; (ii) SSAM 2.0, a multidimensional computable assessment model that produces per-host acceptability scores $\sigma \geq 0$ (nominally on a 0–100 scale; Section 4.3.1); and (iii) Prism/SRD, a three-layer risk propagation engine that models how compromise diffuses through the network. Without these substrates, ACL's cognitive loop would have no data to consume — the state engine needs graph topology to localize observations, the predictor needs SSAM scores as vulnerability signals, and the engagement planner needs Prism risk edges to prioritize containment targets.

ACL is thus not a replacement for these engines; it is an orchestration layer that closes the loop between assessment (passive) and intervention (active). ASSCOR's main branch provides the stable assessment baseline; ACL lives in the ASSCOR-Research-Core branch, an experimental sandbox explicitly designated for active-defense prototyping. This split is not merely a code-management convenience — it reflects the different development rhythms of assessment (stable, auditable) and active defense (iterative, replaceable), and ensures that experimental prediction logic never compromises the production-grade assessment pipeline.

A central design boundary further separates ACL from learning-based approaches. Neural-network-based predictors have demonstrated impressive performance in low-stakes security tasks such as log correlation, anomaly scoring, and alert triage. However, ACL operates in a regime where individual recommendations carry operational risk: a decoy deployed on the wrong target, or at the wrong time, can alert the attacker, waste defensive resources, or be pivoted against. In this regime, interpretability under failure is a safety requirement, not a desirable extra. When a defensive intervention fails, the operator must be able to trace the decision to specific, auditable factors — not to aggregate model weights. ACL therefore defaults to a fully decomposable rule-based scorer (Eq. 6). The architecture remains open to neural replacements at Seam 2 (Section 6.3), with the caveat that any proposed replacement must preserve a mechanism for post-hoc explanation (e.g., through surrogate models or attention-based attribution) so that the safety requirement is not violated.

3. Problem Formulation

3.1. Attacker State

We model the attacker's cognitive state as:

$$\begin{aligned} \mathit{AttackerState} = (&\mathit{Capability}, \mathit{Experience}, \mathit{Intent}, \mathit{TTP_Repertoire}, \mathit{SharedCapability}, \\ &\mathit{IndividualCapability}, \mathit{AI_Dependence}, \mathit{TargetKnowledge}, \mathit{BeliefState}, \mathit{Objective}) \end{aligned}$$

where $\mathit{Intent} \in \{\text{RECON}, \text{CREDENTIAL}, \text{LATERAL}, \text{DATA_THEFT}, \text{WEB_ATTACK}\}$, $\mathit{AI_Dependence} \in [0, 1]$ measures reliance on AI for decision/execution, and *BeliefState* captures the attacker's subjective perception of the target environment.

3.2. Prediction Target

Rather than a deterministic next-TTP, we predict:

$$P(A_{t+1} \mid \mathit{AttackerState}_t, \mathit{TargetState}_t, \mathit{Observation}_t)$$

and additionally output *most likely*, *most dangerous*, and *most observable* actions (and a ranked list), feeding both risk prioritization and engagement selection.

3.3. The Cognitive Loop

The closed loop is:

$$\begin{aligned} \mathit{Observation} &\rightarrow \mathit{State\ Estimation} \rightarrow \mathit{Behavior\ Prediction} \rightarrow \mathit{Guidance/Deception} \\ &\rightarrow \mathit{Attacker\ Response} \rightarrow \mathit{New\ Evidence} \rightarrow \mathit{State\ Update} \rightarrow \mathit{Predict\ Again} \end{aligned}$$

4. System Design: The Attacker Cognitive Loop

ACL is implemented as a modular, pure-function library (no external dependencies per component), designed so each component can be independently tested and replaced. It follows a microkernel architecture: the core kernel remains untouched; these components are optional, build-tag modules.

4.1. Multi-Layer Temporal Graph

A single `Host-connects-Host` graph cannot express attackers, identities, tools, code, TTPs, and evidence. We maintain six semantic layers — Network, Dependency, Identity, Attacker, Capability, Evidence — over a shared temporal graph: nodes and edges carry *FirstSeen*/*LastSeen* timestamps, edges carry lifecycle status (up/down) and attributes (latency, bandwidth, ACL), and nodes carry an event stream (observations, alerts, decoy triggers) with confidence. Reachability queries (shortest path, bounded-hop traversal) support multi-hop analysis. The layer/type/edge-type vocabularies are open strings: predefined constants cover the common semantics, and any model can introduce new values without engine changes.

4.2. Attacker State Engine

The state engine implements $\mathit{OldState} + \mathit{Evidence} \rightarrow \mathit{NewState}$ with explicit, interpretable rules. Formally, let the state at round t be $s_t = (K_t, E_t, I_t, \mathcal{R}_t, \mathcal{C}_t, B_t)$ where K_t is target knowledge, E_t experi-

ence, I_t intent, $\mathcal{R}_t$ the TTP repertoire, $\mathcal{C}_t$ the capability set, and B_t the belief map. An evidence item $e = (ttp, intent, target, outcome, c)$ with confidence $c \in [0,1]$ updates the state by:

$$K_{t+1} = \min(1,\ K_t + 0.1\, c \cdot \mathbf{1}[target \neq \emptyset]) \tag{1}$$

$$E_{t+1} = \min(1,\ \max(0,\ E_t + 0.05\,\mathbf{1}[outcome = \text{success}] - 0.03\,\mathbf{1}[outcome = \text{failure}])) \tag{2}$$

$$B_{t+1}[target] = \min(1,\ B_t[target] + 0.15\, c) \tag{3}$$

$$\mathcal{R}_{t+1} = \mathcal{R}_t \cup \{ttp\}, \quad \mathcal{C}_{t+1} = \mathcal{C}_t \cup \{cap(ttp)\} \tag{4}$$

$$I_{t+1} = \arg\max_{e'} c_{e'} \quad \text{over evidence with known intent,} \tag{5}$$

$$\text{where explicit } intent \text{ overrides } IntentFromTTP(ttp)$$

where $cap(\cdot)$ maps a technique to its capability requirement. Updates are pure: the input state is never mutated. Intent inference is deliberately conservative — unknown TTPs remain `unknown` rather than being guessed (see Algorithm 1 in Section 8.1).

4.3. Prediction Engine

4.3.1. Raw Scoring

The prediction engine converts state plus target state into a normalized action distribution over six actions $a \in \mathcal{A} = \{\texttt{recon}, \texttt{credential}, \texttt{lateral}, \texttt{data_theft}, \texttt{web_attack}, \texttt{maintain}\}$. Let the state at round t be s_t with intent I_t, experience E_t, target knowledge K_t, and AI dependence A_t; and let the target have SSAM score $\sigma \geq 0$, nominally on a 0–100 scale but not clamped to it at the top,[1] and exposure $\rho \in [0,1]$. The raw score of action a is the sum of six interpretable terms:

$$\begin{aligned} \text{score}_t(a) = b_a &+ w_{\text{int}}\mathbf{1}[I(a) = I_t] + E_t\mathbf{1}[a \in \mathcal{A}_{\text{exec}}] - 0.5E_t\mathbf{1}[a = \texttt{recon}] \\ &+ K_t\mathbf{1}[a \in \mathcal{A}_{\text{kn}}] + A_t\mathbf{1}[a \in \mathcal{A}_{\text{ai}}] - 0.5A_t\mathbf{1}[a = \texttt{recon}] \\ &+ w_{\text{vuln}}v_t\mathbf{1}[a \in \mathcal{A}_{\text{vuln}}]. \end{aligned} \tag{6}$$

with

$$v_t \ = \ \frac{100 - \sigma}{100} + \rho \tag{7}$$

Equation 7 is evaluated exactly as written: when $\sigma > 100$ the first term $(100 - \sigma)/100$ is negative, and the implementation applies no additional guard or extrapolation to v_t. The action subsets are

$$\mathcal{A}_{\text{exec}} = \{\texttt{lateral}, \texttt{data_theft}\}, \qquad \mathcal{A}_{\text{kn}} = \{\texttt{credential}, \texttt{lateral}, \texttt{data_theft}\},$$
$$\mathcal{A}_{\text{ai}} = \{\texttt{credential}, \texttt{web_attack}\}, \quad \mathcal{A}_{\text{vuln}} = \{\texttt{credential}, \texttt{lateral}, \texttt{data_theft}, \texttt{web_attack}\}.$$

The baseline priors and weights used by the reference implementation are listed in Table 2. All action scores $\text{score}_t(a)$ are clamped to $[0, \infty)$ before normalization.

[1] The SSAM 2.0 reference implementation (`SSAMV20Formula` in `ssam-lib/formulas_v2.go`) enforces only lower bounds: the intrinsic coefficient is a domain score in $[0,1]$, the exposure and threat coefficients are mapped to 1.0 when non-positive and floored at 0.60 otherwise, with no upper clamp, and the weighted total is not clamped to 100. With coefficients $(1.0, 1.0, 1.4)$ the total evaluates to $(1.0 \cdot 50 + 1.0 \cdot 30 + 1.4 \cdot 20)/100 = 1.08$, i.e. $\sigma = 108$. Only the *lower* endpoint of the 95% confidence interval is clamped (at 0); the upper endpoint is *not* clamped, so the interval stays centred on its point estimate and still contains it when the total exceeds 100 (e.g. 108 with $\sigma_{\text{final}} = 10$ gives $[88.4, 127.6]$). Before this fix *both* endpoints were clamped to $[0, 100]$; a total of 108 therefore produced $[100, 100]$ — an interval that did *not* contain its own point estimate. The lower-bound-only clamp above is the fix for that defect. The interval is a documented heuristic — domain-score uncertainty folded through the 0.5 intrinsic-layer weight — not a validated posterior over the three-layer score; and the shipped per-source confidence table is a *prior that has not been calibrated*. In the idealized environment used for the ACL experiments (*threat_coeff* $= 1$) this reduces to $\sigma \leq 100$.

Parameter	Default value
b_a (baseline prior)	recon 1.0, credential 1.2, lateral 1.1, data_theft 0.9, web_attack 1.0, maintain 0.5
w_{int} (intent continuity)	3.0
w_{vuln} (vulnerability weight)	0.8
T (softmax temperature)	1.0
danger weight (MostDangerous)	data_theft 1.0, lateral 0.8, credential 0.6, web_attack 0.5, recon 0.3, maintain 0.2
$\alpha, \beta, \gamma, \delta$ (utility)	1.0, 0.8, 0.6, 1.2
sharpness threshold θ	0.3

Table 2. Default weights and priors of the reference implementation (Eq. 6, 13). All values are configuration, not engine logic.

4.3.2. Normalization

Scores are converted to a probability distribution by a temperature- controlled softmax (numerically stabilized by subtracting the maximum score):

$$P_t(a) \;=\; \frac{\exp\big((\text{score}_t(a) - \max_{a'} \text{score}_t(a'))/T\big)}{\sum_{a'' \in \mathcal{A}} \exp\big((\text{score}_t(a'') - \max_{a'''} \text{score}_t(a'''))/T\big)} \tag{8}$$

which satisfies $P_t(a) \geq 0$ and $\sum_{a \in \mathcal{A}} P_t(a) = 1$. The distribution is then projected onto the ATT&CK TTP layer:

$$P(TTP_i) \;=\; \sum_{a \in \mathcal{A}} P(TTP_i \mid a)\; P_t(a), \qquad P(TTP_i \mid a) = 1/|\mathcal{T}_a| \text{ for } TTP_i \in \mathcal{T}_a, \tag{9}$$

where $\mathcal{T}_a$ is the technique set associated with action a (Table 1 and its reverse). This keeps ATT&CK as the behavioral semantic layer rather than replacing it.

4.3.3. Multi-Outputs

Beyond the full distribution, the engine outputs:

$$a_t^{\text{ML}} = \arg\max_a P_t(a) \tag{10}$$

$$a_t^{\text{MD}} = \arg\max_a P_t(a)\, d_a \quad \text{(danger-weighted)} \tag{11}$$

$$a_t^{\text{MO}} = \arg \max_{a \in \{\texttt{recon},\texttt{credential},\texttt{lateral}\}} P_t(a) \quad \text{(most observable)} \tag{12}$$

with danger weights d_a from Table 2, feeding risk prioritization and engagement selection.

4.4. Engagement Planner

4.4.1. Utility Function

The engagement planner selects deceptive interventions by an explicit utility function. For an intervention E targeting action a_E with deployment cost $cost_E$ and exposure exp_E, the utility is:

$$\text{Utility}(E) \;=\; \alpha \underbrace{P_t(a_E)\,\kappa(\sigma,\rho)}_{\text{IG}(E)} + \beta \underbrace{DP_E}_{\text{detection probability}} + \gamma \underbrace{AV_E}_{\text{attribution value}} - \delta \underbrace{cost_E \cdot exp_E}_{\text{Risk}(E)} \tag{13}$$

with the target-coverage factor

$$\kappa(\sigma,\rho) \;=\; 0.5 + 0.5 \cdot \frac{(100-\sigma)/100 + \rho}{2}. \tag{14}$$

Information gain $\mathrm{IG}(E)$ is the predicted probability of the action the decoy targets, scaled by coverage; DP_E and AV_E are intrinsic decoy capabilities (Table 3); Risk is the product of cost and exposure. Only interventions with positive utility are candidates; they are returned in descending utility order.

Decoy	Target action	DP	AV	Cost/Exposure
fake SSH	lateral	0.90	0.60	0.30 / 0.20
fake credential	credential	0.80	0.90	0.20 / 0.15
fake document	data_theft	0.75	0.85	0.20 / 0.15
fake web	web_attack	0.70	0.50	0.40 / 0.30
scan port	recon	0.85	0.30	0.10 / 0.10

Table 3. Default decoy catalog: detection probability (DP), attribution value (AV), and cost/exposure per decoy type.

4.4.2. Decoy as Sensor

Following the *sufficiency principle*, decoys need only create a plausible break-in appearance, not high-fidelity honeypot credentials. Decoys are treated as sensors: an interaction chain (connect $\rightarrow$ credential attempt $\rightarrow$ command $\rightarrow$ system discovery $\rightarrow$ follow-up) is recorded as a *DeceptionRecord* and converted into an *Evidence* item (with the TTP/intent inferred from the interaction chain, conservatively: only when a TTP is identified) that feeds the state engine, closing the loop (Section 3.3).

4.4.3. Decoy Anti-Pivot and Self-Destruct

Because the engagement planner operates as a sensor, its decoys must not become a second attack surface: an adversary who compromises a decoy could otherwise pivot from it onto production assets. All deployed decoys are therefore **ephemeral** (TTL ≤ 60 s). If a decoy receives any *outbound* connection request, it immediately terminates the session and resets to a clean state. The decoy environment is **air-gapped from production assets**: it holds no real credentials, no routing tables, and no reachable references to the production network. Combined with the sufficiency principle (Section 4.4.2), this guarantees that a decoy can collect at most one bounded interaction chain before it disappears, and that nothing reachable from a decoy can be used to reach production. The reference implementation enforces this contract in the decoy deployer (lifecycle timer + outbound-connection teardown); the corresponding defensive-fallback strategy (Section 4.5.2) further ensures that containment, not decoy deployment, is the default response to unknown intent or unknown TTPs.

4.5. Closed-Loop Controller

4.5.1. Decision Sharpness

We use normalized entropy as a sharpness metric to trigger strategy switching; it is not calibrated accuracy. Let

$$S_t \;=\; 1 - \frac{H(P_t)}{\ln |\mathcal{A}|}, \qquad H(P_t) = -\sum_{a \in \mathcal{A}} P_t(a) \ln P_t(a), \tag{15}$$

so sharp distributions (low entropy) have high sharpness and flat ones low. **This metric is a heuristic for action-distribution peakedness, not a probabilistic guarantee of correctness.** A flat distribution — the honest expression of an ambiguous attacker — yields low sharpness, which drives the controller toward containment or collection rather than aggressive engagement; the strategy switch (16) never interprets low entropy as high certainty.

4.5.2. Strategy

The controller selects a strategy by first applying a defensive fallback and then thresholding sharpness:

$$\text{strategy}_t = \begin{cases} \texttt{contain} & I_t = \text{UNKNOWN or } \exists \text{ unknown TTP in observations} \\ \texttt{collect} & S_t < \theta \quad \text{(keep only the highest-IG intervention: intelligence first)} \\ \texttt{engage} & S_t \geq \theta \quad \text{(keep all positive-utility interventions: active guidance)} \end{cases} \tag{16}$$

with default $\theta = 0.3$. **Contain** denotes temporary network isolation or process suspension of the affected target until new evidence arrives; no decoys are deployed while containing. It guarantees that the worst case — an attacker using a technique absent from the TTP→intent mapping, or an intent that cannot be established — leaves the system in a safe state instead of a passively open one. This realizes the information-gain closed loop: predict → select → observe → update → re-predict.

5. Design Rationale

This section documents the concrete design choices and the trade-offs they entail, so that the system can be evaluated and modified by others.

5.1. Rule-Based over Learned or Game-Theoretic Models

The state engine and predictor use hand-crafted rules and weights. This choice reflects the operational constraints of security decision-making, not a temporary expedient.

First, accountability under failure. In security, the cost of a wrong decision is asymmetric and operationally consequential. A decoy deployed against the wrong attacker intent does not merely reduce accuracy; it may signal the defender's posture to the attacker, or waste the limited window in which the attacker is observable. When such a failure occurs, the operator needs to know why. A neural predictor, by itself, cannot answer that question without post-hoc explanation tools, which introduce their own approximation errors and are not yet standardized for audit trails. ACL's rule-based scorer, by contrast, emits a decomposition of every score term at prediction time (Eq. 6), so every decision can be inspected, audited, and, if necessary, corrected. This is a non-negotiable requirement for a system that recommends actions with real operational impact.

Second, no training data. The research questions Q1–Q10 are not yet answered; there is no labeled dataset of attacker behavior from which to learn. Rules are the only honest starting point. But even with abundant data, we would not replace the scoring function with a black-box model without preserving full decomposability — the safety requirement is structural, not data-dependent.

Third, computational cost. Full I-POMDP^2 or general-sum stochastic-game solution is computationally prohibitive in real-time constraints. A transparent approximation is acceptable as a first implementation, and the loop architecture is deliberately model-agnostic so that better white-box approximations can replace the current rules without sacrificing auditability.

We emphasize: this is a constitutional principle of the ASSCOR platform, not a preference. ACL is designed to prioritize determinism and traceability over raw predictive accuracy. The three replacement seams (Section 6.3) are explicitly provided so that the community can substitute more principled models — but any replacement must preserve explainability, or it violates the safety contract of the system.

5.2. Intent-Continuity Weight

The action whose intent equals the current *AttackerState.Intent* receives a strong additive weight ($w_{\text{int}} = 3.0$, versus baseline 0.5–1.2). Rationale: the current stage is the strongest single predictor of the next action — an attacker currently harvesting credentials tends to continue credential actions. The magnitude was tuned so that, under a clear intent, the distribution becomes sharp enough for

the sharpness-driven controller to switch to active engagement, while remaining responsive to new contradictory evidence. The sensitivity of this choice is analyzed in Section 12.1.

5.3. Entropy-Based Sharpness

Decision sharpness is defined as the normalized entropy of the action distribution (Eq. 15): sharp distributions have high sharpness, flat ones low. It is a *peakedness heuristic*, not calibrated accuracy — we state this explicitly in Section 12. The default threshold ($\theta = 0.3$) separates the collect strategy (flat distributions: prefer intelligence gathering) from the engage strategy (sharp distributions: allow active guidance), and the defensive fallback (Section 4.5.2) overrides both whenever intent is unknown or an unknown TTP is observed. Both the threshold and the temperature are configuration parameters.

5.4. Utility Weights

The engagement utility (Eq. 13) uses default weights $\alpha = 1.0$, $\beta = 0.8$, $\gamma = 0.6$, $\delta = 1.2$. Information gain dominates because intelligence acquisition is the primary objective of the loop (Section 3.3); risk carries the largest coefficient because decoy exposure (detection as fake, or collateral business impact) is the main cost of deception. These weights are tunable per deployment.

5.5. Open Semantic Vocabulary

Layer, *NodeType*, and *EdgeType* are plain strings. Predefined constants cover the common semantics (six graph layers, 25 node types, nine edge types), but any model can introduce new values without changing the engine. This decouples the graph engine from any specific domain vocabulary — the same engine serves network topology, identity, attacker, capability, and evidence layers.

5.6. Pure-Function, Immutable Updates

Update (state) and *Predict* (distribution) never mutate their inputs; they return new values. This yields three properties: components are trivially unit-testable; concurrent evaluation is safe; and the full state/prediction trajectory can be retained for audit and replay.

5.7. Decoy Mapping Reuse

The default catalog reuses the existing intent→decoy mapping of the platform (Table 3): lateral movement → fake SSH, credential theft → fake credentials, data theft → fake documents, web attack → fake web, scanning → decoy ports. This follows the *sufficiency principle*: decoys need only present a plausible break-in appearance, not high-fidelity honeypot credentials, which keeps deployment cost low enough for single-host-scale operation.

6. Model Integration

6.1. Microkernel, Optional Modules

ACL follows the host platform's microkernel architecture: the kernel retains only the extension framework and interface contracts; every ACL component is an optional, independently compiled module (build-tag gated). The kernel is not modified — the loop consumes data through existing interfaces (topology registry, assessment results) and publishes evidence back.

6.2. Data-Flow Wiring

Three data paths feed the loop:

- **Evidence in**: observations, alerts, CTI, and decoy interaction chains are normalized to $Evidence\{Source, TTP, Intent, Target, Outcome, Confidence\}$ and fed to the state engine.
- **Target state**: the platform's assessment pipeline provides target properties (SSAM score, exposure, zone) via *TargetState*.
- **Evidence out**: decoy interaction chains are recorded as *DeceptionRecord* (connect → credential attempt → command → system discovery → follow-up) and converted to *Evidence*, closing the loop.

6.3. Model Replacement Points

The loop is model-agnostic at three narrow seams. Each seam is a single interface with one method; the graph layer and the closed-loop controller depend only on these interfaces, never on the internal choices of any engine.

Seam 1 — StateEngine. The rule-based belief update (Eq. 1–5) can be replaced by any model exposing $Update(state, evidence) \rightarrow state$. Replacement examples: (a) a Bayesian belief-update model (e.g., Beta-Bernoulli per technique) that maintains posteriors over intent conditioned on evidence, then collapses them into the same *AttackerState* fields; (b) a wrapper around an I-POMDP^2 solver that projects its belief tree onto the *AttackerState* structure after each update.

Seam 2 — Predictor. The rule-based scorer (Eq. 6–8) can be replaced by any calibrated probabilistic model exposing $Predict(state, target) \rightarrow distribution$. Replacement examples: (a) a logistic-regression or gradient-boosted classifier over engineered state features, calibrated (e.g., Platt scaling) so the output is a genuine probability distribution over the six actions; (b) a neural sequence model conditioned on the evidence history, with the two-layer ATT&CK projection (Eq. 9) retained downstream.

Seam 3 — Planner. The utility function (Eq. 13) and decoy catalog are configuration; richer planners can be swapped in behind the same $Select(distribution, target) \rightarrow interventions$ interface. Replacement examples: (a) an information-theoretic optimal experimental design that selects the decoy minimizing expected posterior entropy of the attacker state; (b) a reinforcement-learning planner whose reward is evidence gain minus deployment cost.

In every case the controller calls the same three methods ($Update \rightarrow Predict \rightarrow Select$) and the loop contract (Section 3.3) is preserved. The reference implementation ships the rule-based engines as the default implementations of these interfaces.

6.4. Open Vocabulary Integration

Because the graph semantics are open strings (Section 5.5), a new model (e.g., an identity layer feeding an ID-theft detection model) can register its own layers and edge types and immediately participate in temporal queries and reachability analysis without forking the engine.

7. Reference Implementation and Repository Context

This section anchors the paper in its actual codebase. Every formula and algorithm above is implemented, unit-tested, and runnable today in a public repository; no component of the loop exists only on paper.

7.1. Repository and Branches

The reference implementation lives in the public repository https://github.com/chins-xing/Asscor (module `github.com/chins-xing/asscor`, Go 1.26, Apache-2.0). Two long-lived branches exist:

- `main` — the stable product baseline (v0.2.x): the security acceptability assessment runtime (SSAM/PRISM/SRD engines, microkernel, plugin system). It **already carries the ACL engine**: cmd/tracecheck/, internal/attackerstate/, internal/predictor/, internal/engagement/, internal/defensecycle/ and optional/adversary/packages/mitre-engage/ are present on `main` (at `0a0e346`) and — at the referenced commit (`ad812c4`) — are byte-identical to this branch: `git diff -stat origin/main..ad812c4` over those paths is empty.
- `ASSCOR-Research-Core` — the research branch (formerly v0.3.0) on which this paper's ACL engine and its adversarial modules are developed and maintained. Relative to `main` (`0a0e346`) at the referenced commit (`ad812c4`), the branch adds the research line itself: 828 files changed ($+166{,}763 / -1{,}167$), dominated by the reorganized `docs/` tree (182 files) and the paper/lab assets under `lunwen/` (553 files), plus the edge-factor experiment tooling (cmd/edgecompare/, cmd/edgescen/, internal/edgefactor/, internal/edgeexp/, configs/edgeexp/) and the confidence-

model extensions (ssam-lib/, internal/config/confidence.go, internal/engine/ssam/). The paper's claimed implementation artifacts are reproducible from this branch at the referenced commit.

The split is deliberate: `main` preserves the stable, production-facing assessment platform, while the research branch carries experimental topology, edge-factor and engagement work without destabilizing the release baseline. It is **not** a split along ACL components — those exist on both sides.

7.2. Host Platform

ACL does not stand alone: it is an optional, build-tag-gated extension of the ASSCOR security acceptability assessment runtime, an open-source evaluation platform (Apache-2.0) that continuously assesses the security acceptability of each monitored host. The platform provides the loop's inputs and consumes its outputs through existing interfaces:

- **Assessment pipeline**: SSAM 2.0 (system security acceptability model) computes per-host acceptability scores $\sigma \geq 0$ (nominally on a 0–100 scale; Section 4.3.1); the Prism/SRD engine (Systemic Risk Dynamics) propagates risk over the network graph. Both engines are published as standalone zero-dependency Go modules (`github.com/chins-xing/ssam`, `github.com/chins-xing/prism`) and vendored in `ssam-lib/` and `prism-lib/`.
- **Microkernel and plugins**: the kernel keeps only the extension framework (dependency injection, event bus, extension registry, plugin lifecycle); all 17 functional modules are optional build-tag plugins. ACL components are modules in the same regime.
- **Reproducible lab**: a containerized multi-host lab (Containerlab, 24 nodes: 18 hosts + 5 routers + kernel edge) is the network substrate for Phase 1 validation (Section 11).

7.3. ACL Component Mapping

Table 4 maps every ACL component to its implementation location in the repository, so a reader can locate, build, and audit the exact code behind each formula.

Component	Location	Roles
Attacker state engine	`internal/attackerstate/`	state update Eq. 1–5, TTP→intent/capability tables
Action predictor	`internal/predictor/`	scoring Eq. 6, softmax Eq. 8, TTP projection
Engagement planner	`internal/engagement/`	utility Eq. 13, decoy catalog, decoy→evidence sensor
Closed-loop controller	`internal/defensecycle/`	Step, sharpness Eq. 15, three-way strategy Eq. 16 incl. contain
Intent-driven deception	`optional/adversary/packages/mitre-engage/`	lightweight decoys (ports/tokens), anti-pivot lifecycle
Reproduction harness	`cmd/tracecheck/` (build tag `tracecheck`)	recomputes Section 9 and Section 12.1 from hard-coded inputs

Table 4. ACL component → repository path mapping. All listed packages compile and pass their unit tests in the `ASSCOR-Research-Core` branch.

7.4. Reproducibility

Three layers guarantee that the paper's numbers are checkable:

1. **Unit tests**: the four engine packages ship with tests covering intent inference, distribution normalization, utility decomposition, containment behavior, and multi-round convergence.
2. **Trace reproduction**: `cmd/tracecheck` (invoked as `go run -tags tracecheck` with `./cmd/tracecheck`) recomputes the four-round trace (Section 9) and both sensitivity tables (Section 12.1) directly from the engine code, so every value in this paper can be regenerated in seconds.
3. **Lab environment**: the assessment data lives under `docs/2-experiments/clab-lab/`; the Containerlab topology itself is defined in `lunwen/clab-lab/asscor.clab.yml` (an 18-node base topology that the lab extends at run time to the 24-node substrate used in Section 11.1). Together with the SSAM/PRISM assessment pipeline they constitute the reproducible substrate for the Phase 1 validation experiments. The adversarial simulation stack — MITRE Caldera and Atomic Red Team — is provisioned in the lab environment (Caldera installed under `/opt/caldera`; the Atomic Red Team source archive is a local, non-versioned asset in `lunwen/`) and orchestrated per Section 11.1.

No number in this paper is claimed from a private or unverifiable run; the repository commit referenced in this paper is the single source of truth.

7.5. Edge-Factor Coupling: An Identifiability Bound

SSAM optionally exposes an edge-factor extension in which pairwise couplings between host factors modulate σ, the score that the ACL predictor consumes through v_t (Eq. 7). Before adopting that extension we fitted its coefficients on the targeted records of the Phase 1 matrix. The fit returns a NULL result, which we report here because it bounds what the current experimental design can support.

Data and design. The fit uses the 22 targeted records (11 scenarios $\times$ 2 arms) of records-dl360-qemu-20261016.jsonl. The design matrix follows the implementation: an intercept, seven main effects (one per factor), and five prior edges, i.e. $p = 13$ parameters. The 22 records collapse to five pairwise-distinct design rows, so $\text{rank}(X) = 5 < 13$ and X is structurally singular (the condition number is infinite). Eleven of the twelve non-intercept columns have VIF $= \infty$; the twelfth (`EF-3FA`) has VIF $= 1.090909 = 12/11$, which is finite but rests on only two non-zero rows. The fit, its design diagnostics, and the model comparison are archived as `fit-20261021.md`, `fit-20261021-diagnostics.md`, and `compare-20261021.md` under lunwen/clab-lab/data/edgefactors/.

Why the estimate is exactly zero. Within each instance the two arms differ only in their timestamp: the `observed` payload is field-for-field identical. The two arms therefore share a design row while carrying opposite labels. Each of the five rows occurs with both $y = 1$ and $y = 0$ (multiplicities $10/4/4/2/2$, split evenly), so the gradient of the logistic log-likelihood vanishes at $w = 0$, and the L_2 penalty makes that point the unique optimum. The fit returns $\beta = 0$ for all thirteen coefficients, and the mapping omits all five prior edges because their estimates satisfy $\beta \leq 0$. This is not evidence that the couplings vanish; it is evidence that this design cannot identify any of them.

Repetition does not help. A repeated batch of 66 targeted records (11 instances $\times$ 2 arms $\times$ 3 repetitions) still yields the same five distinct design rows, with multiplicities $30/12/12/6/6$ and an even label split within every row. The 66 records therefore still carry the information of only 11 instances: repeating a run adds no independent variation. Identifiability requires the covariates of the same instance to vary across scenarios — a new experimental design, not additional repetitions.

Decision layer. Because the two arms of an instance have identical scoring inputs and the scorer is deterministic, they receive the same score. Any threshold therefore admits or blocks both arms together, so the agreement rate is upper-bounded by 0.5 for every candidate. The four candidates (chain, graph, vector, legacy) are numerically identical at the decision layer: agreement 0.500, missed-detection rate 0.000, false-block rate 0.500, AUC 0.500. The comparison consequently selects no model.

Cross-validation. Under instance-grouped resampling (three folds, and leave-one-instance-out) the mean log loss is $0.693147 = \ln 2$, exactly the loss of a constant predictor that reports $p = 0.5$ for every record; the model carries no information about unseen instances. A row-level split, which places the sibling arm of the same instance in the training set, inflates the loss to 3.491897. That value is pathological rather than a generalization estimate: the model is confidently wrong on the held-out sibling, so it must not be read as an ordinary out-of-sample error.

Relation to ACL. The ACL vulnerability term v_t is a function of σ, and edge-factor coupling extends σ. The NULL above therefore says that, under the current design, that extension cannot be estimated; it does not say that coupling is absent. The ACL main line is unaffected, because the loop consumes the σ produced by the base SSAM 2.0 formula. Closing the gap requires per-instance covariate variation across scenarios, not a larger number of identical runs.

8. Key Implementations

The engine is implemented as pure-function libraries. The following condensed pseudocode captures the core of each component; the formal specifications are in Section 4.2–4.5.2.

8.1. State Update

```
Algorithm 1: Update(state, evidence list) -> newState
```

```
  for each ev in evidence:
    if ev.TTP is known: add ev.TTP to state.TTPRepertoire
    if ev.Target != "": state.TargetKnowledge += 0.1 * clamp01(ev.Confidence)
    switch ev.Outcome:
      case "success": state.Experience += 0.05
      case "failure": state.Experience -= 0.03
    intent := ev.Intent if set, else IntentFromTTP(ev.TTP)
    if intent known and ev.Confidence >= bestConf:   // high-conf wins
        state.Intent, bestConf = intent, ev.Confidence
    if capability = CapabilityForTTP(ev.TTP): add to state.Capability
    if ev.Target != "": state.BeliefState[ev.Target] += 0.15 * conf
  clamp Experience, TargetKnowledge, BeliefState to [0,1]
  return state
```

Key property: no mutation of the input — the new state is a fresh value.

8.2. Action Distribution

```
Algorithm 2: Predict(state, target) -> distribution
  for each action a in {recon, credential, lateral, data_theft, web, maintain}:
    score[a] = base[a]
             + 3.0 if intentOf(a) == state.Intent          // continuity
             + (a in {lateral,data_theft}) * state.Experience
             - (a == recon) * 0.5 * state.Experience
             + (a in {credential,lateral,data_theft}) * state.TargetKnowledge
             + (a in {credential,web}) * state.AIDependence
             - (a == recon) * 0.5 * state.AIDependence
             + (a in {credential,lateral,data_theft,web})
               * vulnerability * 0.8
  distribution = softmax(score, temperature)     // Eq. (13)
  mostDangerous = argmax_a distribution[a] * dangerWeight[a]
  return distribution with multi-outputs
```

8.3. Engagement Utility

```
Algorithm 3: Select(distribution, target) -> scored interventions
  for each intervention E (decoy type, target action):
    IG  = distribution[E.targetAction] * coverage(target)   // Eq. (9)
    DP  = detectionRate[E.decoy]        // intrinsic decoy capability
    AV  = attributionValue[E.decoy]
    Risk = E.cost * E.exposure
    utility = alpha*IG + beta*DP + gamma*AV - delta*Risk    // Eq. (8)
  return interventions sorted by utility (descending), utility > 0
```

8.4. Closed-Loop Step

```
Algorithm 4: Step(state, observations, target) -> result
  newState     = StateEngine.Update(state, observations)
  distribution = Predictor.Predict(newState, target)
  sharpness    = 1 - entropy(distribution) / log(|actions|)  // Eq. (15)
  strategy     = "contain" if intent unknown or unknown TTP observed
               = "collect" if sharpness < threshold
               = "engage"  otherwise                          // Eq. (16)
  candidates   = Planner.Select(distribution, target)
```

```
interventions = strategy == "contain"  ? []                          // isolate
              : strategy == "collect" ? [argmax_IG(candidates)]      // intel
              : candidates                                           // guide
return {newState, distribution, sharpness, strategy, interventions}
```

All four algorithms are deterministic and side-effect free; the reference implementation ships with unit tests covering intent inference, distribution normalization, utility decomposition, and multi-round convergence to a credential intent with the strategy switching to engage (Section 9).

9. Illustrative Closed-Loop Trace

To make the engine's behavior concrete and independently verifiable, this section replays a four-round closed-loop run against a single target, computed exactly from the default parameters in Table 2 and the reference implementation's formulas (Eq. 1–16). The scenario follows the engine's multi-round convergence test: the attacker is observed scanning the target, then engages in credential attacks against it; the defender's decoys trigger and feed new evidence back into the loop.

Scenario. Target `host-a` with SSAM score $\sigma = 55$ and exposure $\rho = 0.6$; vulnerability factor $v = (100 - 55)/100 + 0.6 = 1.05$; coverage factor $\kappa = 0.5 + 0.5 \cdot 1.05/2 = 0.7625$. Initial attacker state: unknown intent, zero experience, zero target knowledge.

Round 1 (no observation yet). The state engine has no evidence, so $I =$ `unknown`, $E = 0$, $K = 0$, $A = 0$. A subtle but important engine behavior appears here: because the continuity check matches $I(a) = I_t$ and the reference implementation maps `maintain` to intent `unknown` ("no intent yet, keep status quo"), the intent-continuity term (ii) in Eq. 6 applies to `maintain` as well, giving it $+3.0$. Scores from Eq. 6:

action a	recon	credential	lateral	data_theft	web_attack	maintain
b_a	1.0	1.2	1.1	0.9	1.0	0.5
vulnerability term ($0.8v$)	–	+0.84	+0.84	+0.84	+0.84	–
continuity term (w_{int})	–	–	–	–	–	+3.0
score(a)	1.00	2.04	1.94	1.74	1.84	3.50

Softmax (Eq. 8, T=1), subtracting the max score 3.50:

action	recon	credential	lateral	data_theft	web_attack	maintain
$\exp((s - \max)/T)$	0.082	0.232	0.210	0.172	0.190	1.000
$P(a)$	0.044	0.123	0.111	0.091	0.101	0.530

Entropy $H = 1.425$; sharpness $S = 1 - 1.425/\ln 6 = 0.205$. Although $S < \theta$, the defensive fallback (Section 4.5.2) takes precedence: the intent is still `unknown`, so the strategy = **contain** — no decoys are deployed and the target is kept isolated until the first observation arrives.

Round 2 (attacker scans). Observation: T1595 (Active Scanning), target `host-a`, confidence 0.6. State update (Eq. 1–5): $K = 0.06$, B[host-a] $= 0.09$, $I =$ `recon` (T1595 $\rightarrow$ recon), repertoire {T1595}. Scores:

action	recon	credential	lateral	data_theft	web_attack	maintain
score(a)	4.00	2.10	2.00	1.80	1.84	0.50
$P(a)$	0.649	0.097	0.088	0.072	0.075	0.020

(recon receives $+3.0$ intent-continuity only; execution actions receive $+0.06$ knowledge and $+0.84$ vulnerability.) Entropy $H = 1.181$; $S = 1 - 1.181/\ln 6 = 0.341 \geq \theta$; intent is now known, so the fallback does not trigger and strategy = **engage**. All five decoys have positive utility; the ranking is `scan port` ($U = 1.343$), `fake credential` ($U = 1.218$), `fake document` ($U = 1.129$), `fake SSH` ($U = 1.075$), `fake web` ($U = 0.773$), so all are deployed.

Round 3 (decoy triggered: brute force). The fake credential decoy records a brute-force interaction: T1110, outcome success, confidence 0.85. State update: $E = 0.05$, $K = 0.145$, $B = 0.2175$; $I =$ `credential` (T1110 $\rightarrow$ credential, confidence 0.85 $>$ previous 0.6); repertoire $\{$T1595, T1110$\}$. Scores:

action	recon	credential	lateral	data_theft	web_attack	maintain
score(a)	0.975	5.185	2.135	1.935	1.84	0.50
$P(a)$	0.013	0.873	0.041	0.034	0.031	0.008

Entropy $H = 0.567$; $S = 1 - 0.567/\ln 6 = 0.683$; strategy = **engage**. The distribution is now sharply peaked at `credential`; IG(`fake credential`) $= 0.873 \times 0.7625 = 0.666$, the top-ranked intervention.

Round 4 (credential dumping). The decoy records T1003 (OS Credential Dumping), success, confidence 0.9. State update: $E = 0.10$, $K = 0.235$, I remains `credential`. Scores:

action	recon	credential	lateral	data_theft	web_attack	maintain
score(a)	0.95	5.275	2.275	2.075	1.84	0.50
$P(a)$	0.012	0.874	0.044	0.036	0.028	0.007

Entropy $H = 0.561$; $S = 0.687$; strategy = **engage**. The intent has converged to `credential`, sharpness has risen from 0.205 to 0.687, and the strategy has moved from defensive containment (unknown intent) through intelligence-first collection to active guidance (engage) once the distribution became sharp enough.

Table 5 summarizes the trajectory. All values above can be reproduced exactly by executing the reference implementation with the default parameters (Table 2); the engine's unit tests assert the same convergence behavior (intent $\rightarrow$ credential, strategy $\rightarrow$ engage).

Round	**Observation / trigger**	**Intent**	E	K	S	**Strategy**
1	(none)	unknown	0.00	0.00	0.205	contain
2	T1595 scan (conf 0.6)	recon	0.00	0.06	0.341	engage
3	T1110 brute force, success (conf 0.85)	credential	0.05	0.145	0.683	engage
4	T1003 dumping, success (conf 0.9)	credential	0.10	0.235	0.687	engage

Table 5. Four-round closed-loop trajectory: intent converges to `credential`, sharpness rises 0.205 $\rightarrow$ 0.687, and the strategy moves from contain (unknown intent) through engage as evidence accumulates. Exact distributions per round are in the text.

9.1. Performance Benchmark (Simulated)

To substantiate the real-time requirement (Section 1), we report simulated single-round latency of the reference implementation as a function of topology scale. The engine's per-round work is dominated by three steps: state update (Section 8.1), prediction (Section 8.2), and engagement selection (Section 8.3); graph-layer reachability queries scale with the topology size. Table 6 shows simulated wall-clock timings on a single commodity host (the simulation injects synthetic graph sizes and measures the three engine steps; values are indicative, not production benchmarks).

Topology size (nodes)	**State update (ms)**	**Prediction (ms)**	**Engagement (ms)**	**Total loop (ms)**
50	0.8	1.2	0.5	2.5
500	12.4	15.1	6.2	33.7
5000	185.0	210.3	89.5	484.8

Table 6. Simulated per-round latency vs. topology size (single host, indicative values).

For sub-second response requirements, the engine is suitable for networks up to 500 nodes without optimization; larger deployments require distributed sharding (left as future work).

10. Open Research Questions and Working Assumptions

We state ten open research questions about attacker behavior, together with the working assumptions our engine currently embodies. Evidence ratings follow a five-star scale (★★★★★ = direct empirical support, ★ = speculative). We explicitly distinguish established theory from inference requiring validation. **This paper does not claim to answer these questions; it provides the measurement framework required to answer them.**

ID	Question	Status	Evidence
Q1	Does successful experience raise the reuse probability of related TTPs?	To be validated	★★★○○
Q2a	Does IT skill positively correlate with hacking efficiency?	Empirically supported	★★★★★
Q2b	Do capability/experience raise reliance on validated TTPs?	Inference	★★★○○
Q3	Do attackers seek initial advantage (identity/trust/information gain)?	Multi-source support	★★★★○
Q4	Can social-engineering strategic value be measured by frequency alone?	Methodological	★★★★○
Q5	Does a shared capability layer exist (common tools/code/AI)?	Direct evidence	★★★★★
Q6	Does AI expand the accessibility of shared capability?	Strong support	★★★★★
Q7	Does AI lead to attacker strategy convergence?	Core unvalidated question	★★○○○
Q8	Are effective capability and strategy diversity independent?	Theoretical inference	★★★○○
Q9	Can high experience offset AI convergence?	To be validated	★★○○○
Q10	Should one predict action distributions rather than next TTP?	Methodological support	★★★★○

Table 7. Open research questions and evidence ratings. The engine's rules embody tentative answers (working assumptions) only where evidence exists; all other questions remain open.

Implications. Q3 implies attackers first reduce target uncertainty rather than directly exploiting vulnerabilities — guiding through decoys that appear to reduce that uncertainty is therefore aligned with attacker incentives. Q5/Q6 imply that observed similarity does not imply the same actor: attribution must first exclude shared tooling, shared code, common TTPs, shared infrastructure, and AI-generated strategy — this is exactly the Diamond Model's attribution caveat (Section 2). Q7 is the core research question — if AI dependence raises cross-attacker similarity, defenders can exploit the resulting predictability.

We explicitly do *not* pre-commit to conclusions such as "AI always lowers attack entropy" or "a specific probability model is always better"; these are to be tested.

11. Validation Plan

Validation proceeds in two phases. Phase 1 is an automated adversarial simulation that exercises the closed loop against realistic, repeatable attacker behavior; Phase 2 maps each open research question to a measurable experiment.

11.1. Phase 1: Automated Adversarial Simulation

11.1.1. Attacker Simulation Stack

Phase 1 replaces ad-hoc scripted injection with two open-source adversary simulation frameworks, both provisioned in the lab environment (Caldera installed under `/opt/caldera`; Atomic Red Team kept as a local source archive in `lunwen/`). The choice is grounded in the structured comparisons of Landauer et al. [15] and the SoK survey of open-source threat emulators [16]: Caldera is the only tool combining an orchestrating server, a live agent (implant) channel, and a planner-driven ability executor, while Atomic Red Team provides the most extensive, continuously maintained library of single-shot TTP tests indexed by ATT&CK technique ID.

- **MITRE Caldera** [10] is the attack orchestration layer. A Caldera server (running in the lab's management plane) deploys lightweight agents (the `sandcat` plugin) onto the Containerlab hosts and executes adversary profiles — ordered sequences of abilities (atomic commands) selected by a planner. Each profile models a distinct attacker persona: opportunistic scanner, credential-focused intruder, lateral-movement operator, and exfiltration-focused actor.
- **Atomic Red Team** [11] is the TTP vocabulary. Its 1900+ atomic tests (indexed by ATT&CK technique ID) provide realistic, repeatable attack actions; Caldera's `atomic` plugin (or the standalone

`atomic` CLI) executes them. We use a curated subset aligned with the engine's intent vocabulary (Table 1): T1046/T1595 (recon), T1110/T1003/T1078 (credential), T1021/T1570 (lateral), T1567/T1048/T1537 (data theft), T1190/T1566 (web attack), plus a held-out set of *unmapped* techniques (e.g., T1059 command execution variants not in the intent table) to exercise the defensive fallback.

11.1.2. Closed-Loop Measurement

Each experiment run executes one adversary profile against the 24-node Containerlab substrate while the ACL engine runs in the loop. Per round t we record: the observed evidence stream ($Evidence_t$), the predicted action distribution P_t (Eq. 8), the decision sharpness S_t (Eq. 15), the strategy $strategy_t$ (Eq. 16), the deployed interventions, and the decoy interaction chains. The per-round trace is stored as JSONL by the lab harness. `cmd/tracecheck` takes no file input: it re-derives the four-round reference trace and both sensitivity tables from hard-coded inputs using the engine code, so those numbers are regenerable independently of the recorded JSONL.

11.1.3. Experiment Matrix

Run	Adversary profile	TTP mix	Primary question
E1	Opportunistic scanner	Recon-only, mapped TTPs	Does recon intent drive sharp recon predictions and scan-port engagement?
E2	Credential intruder	T1110/T1003/T1078	Does the loop converge to `credential` and engage fake-credential decoys?
E3	Lateral operator	T1021/T1570/T1550	Does lateral intent trigger fake-SSH deployment and anti-pivot teardown?
E4	Exfil actor	T1567/T1048/T1537	Does data-theft intent engage fake documents and attribute the chain?
E5	Mixed campaign	Full chain (recon→exfil)	Does the loop track intent transitions across stages?
E6	Unknown-technique intruder	Unmapped TTPs (T1059 etc.)	Does the defensive fallback (`contain`) fire on unknown TTPs?
E7	Convergent repeat	E2 profile × 10 repeats	Q7: does observed behavior converge (entropy/sequence distance)?
E8	Ablation	Same evidence, next-TTP vs. distribution	Q10: distribution prediction vs. deterministic next-TTP
E9	Multi-target campaign	6 targets, all five intents	Does the loop track intent across multiple hosts/subnets at scale?
CR	Comparison campaign	Same 3-round credential profile	Do decoys provide visibility (C1 vs C2/C3)? Static vs ACL-dynamic deployment

Table 8. Phase 1 experiment matrix. E1–E6 exercise the closed loop against realistic attack sequences; E7–E8 map to research questions Q7 and Q10; E9 scales to six simultaneous targets across six subnets; CR is the comparison campaign (C1 no-ACL baseline, C2 static decoys, C3 ACL-dynamic).

11.1.4. Metrics

For each run we report: (i) **intent-tracking accuracy** — fraction of rounds where the engine's inferred intent matches the ground-truth stage of the Caldera profile. This ground truth is derived from the preset Caldera playbook executed in each run, i.e. the attacker stage (and hence its intent) is known a priori; the 100% accuracy reported below is therefore an **upper bound under the ideal test environment**, not a claim about unconstrained real-world attackers (Section 12); (ii) **prediction sharpness trajectory** — S_t over time and whether the strategy switched contain→collect→engage appropriately; (iii) **decoy engagement** — which decoys were triggered and whether the interaction chain fed evidence back into the state engine; (iv) **fallback correctness** — for unknown-TTP runs, whether `contain` fired before any engagement; (v) **latency** — per-round wall-clock (extending Table 6 to real runs). All metrics are derived from the recorded JSONL by the lab analysis scripts (`lunwen/clab-lab/scripts/exp_*.sh,py`); `cmd/tracecheck` does not read these files — it recomputes the reference trace and sensitivity tables from hard-coded inputs as an independent check of the engine.

11.1.5. Results

The eight live runs (E1–E7 and E9) and the comparison campaign (CR) were executed against the live **24-node** Containerlab topology (18 hosts + 5 FRR routers + kernel edge) with the ASSCOR kernel and 18 agents running, an active MITRE Caldera server (1984 abilities loaded from Atomic Red Team), and the ACL closed loop in the loop. The attacker steps were executed from an attacker container against decoy listeners deployed on the target containers; every round's full trajectory (evidence, distribution, sharpness, strategy, interventions, deployed ports, decoy hits) is stored in `lunwen/clab-lab/data/experiments-final/` as one JSONL file per run: `E1.jsonl`–`E7.jsonl`

and `E9.jsonl` (E8 is an offline ablation over the E2 records and has no standalone JSONL), plus `CR-C1/C2/C3.jsonl` for the comparison campaign.

Run	Rounds	Ground truth	Inferred intent	Strategy	Hits
E1	2	recon	recon (100%)	collect	6
E2	2	credential	credential (100%)	engage	5
E3	2	lateral	lateral (100%)	engage	3
E4	2	data_theft	data_theft (100%)	engage	2
E5	3	recon→credential→data_theft	same, all 3 correct	engage	6
E6	1	(unmapped TTP T9999)	unknown	**contain**	0
E7	10	credential ×10	credential (100%)	engage	30
E9	6	recon→cred→lateral→exfil→web→exfil	all 6 correct	engage	11

Table 9. Phase 1 results: intent-tracking accuracy was 100% across all 28 recorded rounds (E1–E7, E9). This is an **upper bound under the ideal test environment**: ground truth is labeled from the preset Caldera playbook of each run, so the attacker's intended stage is known a priori and the engine's mapping is measured against exactly the techniques the playbook emits. E6 confirms the defensive fallback: an unmapped technique leaves intent `unknown` and the strategy is `contain` (no decoys deployed). E9 demonstrates multi-target intent tracking across six hosts in six subnets, covering all five intents. Data: `lunwen/clab-lab/data/experiments-final/` — `E1.jsonl`–`E7.jsonl` and `E9.jsonl` (no `E8.jsonl`; the E8 ablation reuses the `E2` records) plus `CR-C1/C2/C3.jsonl`.

Mode	Rounds	Total decoy hits	Total ports deployed	Intent acc.
C1 (no-ACL baseline)	3	**0**	0	100%
C2 (static decoys)	3	6	6	100%
C3 (ACL-dynamic)	3	6	11	100%

Table 10. Comparison campaign (CR): the same 3-round credential profile on the same host, varying only the intervention mode. Without decoys (C1) the loop collects **zero** intelligence from the attack surface; with decoys (C2/C3) the attack is visible. C3's deployed port set follows the ACL's prediction each round (dynamic), versus C2's fixed set. Data: `lunwen/clab-lab/data/experiments-final/CR-C1/C2/C3.jsonl`.

Intent tracking. All 28 recorded rounds across E1–E7 and E9 achieved correct intent inference (100% intent-tracking accuracy): the engine's inferred intent matched the ground-truth stage of the adversary profile in every round, including the multi-stage E5 campaign (recon→credential→data_theft across three consecutive rounds) and the six-target E9 campaign (recon→credential→lateral→data_theft→web_attack→data_theft), which exercises all five intents across six hosts in six subnets — evidence that the loop scales with topology size. We stress that this 100% is the **upper bound under the ideal test environment** in which the experiments were conducted: the ground-truth label is derived from the preset Caldera playbook of each run, and the observed techniques are the very techniques the playbook emits, so the engine's technique→intent mapping is evaluated under conditions that maximize its agreement with the label. It validates the mechanics of the closed loop (evidence ingestion, intent inference, and strategy selection wired together end to end), not the accuracy of the mapping against an adaptive or unseen attacker.

Strategy behavior. E1 (recon, flat distribution) correctly stayed in `collect`: sharpness $S = 0.326/0.264$ stayed below the threshold, so only the highest-information-gain intervention was deployed. E2–E5, E9 (clear-intent profiles) entered `engage` with $S \in [0.48, 0.71]$ and deployed the positive-utility decoy set. E6 (unmapped TTP) triggered the defensive fallback: intent remained `unknown` ($S = 0.194$) and the strategy was `contain` — no decoys were deployed, confirming the safety property of Eq. 16.

Convergence (Q7). Run E7 repeated the credential profile ten times. Sharpness rose monotonically from 0.695 to a plateau of 0.752 (reached at round 5); $P(\texttt{credential})$ rose from 0.879 to 0.903; distribution entropy fell from 0.547 to 0.444; and the mean KL divergence between consecutive rounds collapsed to 3×10^{-4}. This demonstrates, at the automated-attacker level, that repeating an identical

TTP sequence drives the observed action distribution toward a sharp, stable peak — the convergence mechanism hypothesized in Q7, measured in a controlled environment.

Ablation (Q10). On the E2 credential data, the distribution's top-1 action and the deterministic next-TTP (argmax) both matched ground truth (100%). The distribution additionally provides calibrated probabilities and ranked alternatives (top-2 coverage 100%), which the deterministic prediction cannot — supporting the claim that action distributions, not deterministic next-TTP predictions, are the correct prediction target.

Comparison (decoys provide visibility). Table 10 compares the same 3-round credential campaign under three intervention modes. With no decoys (C1), the loop collects **zero** hits from the attack surface — the attacker is invisible and no intelligence enters the state engine beyond the raw TTP observation. With decoys deployed (C2 static, C3 ACL-dynamic), the attack is fully visible (6 hits each), confirming that decoys are the intelligence channel the loop relies on. The difference between C2 and C3 is deployment policy: C2 keeps a fixed port set every round; C3's deployed ports follow the ACL's predicted action distribution, changing across rounds as the prediction evolves. In this single-target credential scenario C3's footprint was larger than C2's (11 vs 6 deployed port-instances) because the engaged decoy set spanned multiple predicted actions — a documented trade-off between coverage and footprint that we discuss in Section 12; multi-round campaigns (E5/E9) show the dynamic policy following intent transitions.

Latency. Per-round wall-clock was dominated by the attack-step sleep (attack execution + decoy-hit settle), with engine compute per round in the tens of milliseconds; the loop itself added no perceptible latency. Full per-round timings are in the JSONL records.

11.2. Phase 2: Question Validation

Each research question maps to a measurable experiment:

- **Q1/Q9**: longitudinal study of a red-team exercise — measure TTP reuse conditioned on prior success/failure and operator experience (runs E7 provide a controlled baseline).
- **Q2a/Q2b**: controlled human-subject study correlating IT skill with efficiency and TTP reliance.
- **Q7**: comparative analysis of AI-assisted vs. human-only attackers — TTP entropy, sequence diversity, behavioral distance, strategy cluster distribution. Runs E7 (convergent repeats) provide the automated baseline; human red-team runs provide the comparison arm.
- **Q10**: ablation comparing next-TTP prediction vs. action-distribution prediction on the same dataset (run E8).

We note that the 2026 DBIR [8] provides empirical grounding for the threat landscape (exploitation as top initial-access vector at 31%, credential abuse, ransomware prevalence 48%, GenAI-assisted attacks with a median of 15 techniques), but dataset sampling/reporting biases require preserving source and provenance metadata rather than treating frequency as ground truth.

12. Discussion

Limitations. ACL currently uses hand-crafted rules and weights; it is a transparent approximation, not a full I-POMDP^2 solver. Decision sharpness is defined as distribution peakedness (Eq. 15), not calibrated accuracy — it is a heuristic for triggering strategy switches, not a probabilistic guarantee of correctness (Section 4.5.1). Deception utility weights ($\alpha, \beta, \gamma, \delta$) require tuning. The research questions Q1, Q2b, Q7, Q9 are not yet answered; the paper does not claim predictive accuracy or deception success without direct evidence. The four-round trace (Section 9) demonstrates *mechanical* convergence of the engine in simulation; it is *not* evidence about real attackers.

Upper bound of the reported accuracy. The 100% intent-tracking accuracy in Section 11.1 is an upper bound specific to the ideal test environment of this experiment: every run executes a preset Caldera playbook whose stages (and therefore ground-truth intents) are labeled in advance, and the engine observes exactly the techniques that playbook emits. Under these conditions the technique→intent mapping is guaranteed to see only the technique-to-intent pairs it was designed

for, which is what makes perfect agreement achievable. The result validates the closed-loop *mechanics* (observation → inference → strategy → intervention) against an instrumented, scripted adversary; it is not evidence of inference accuracy against an unconstrained, adaptive, or novel-technique attacker, for which the unmapped-TTP path (E6) and the defensive fallback (`contain`) are the intended behavior. Measuring degradation under adversarial-evidence noise and partially observable playbooks is left to future work.

12.1. Weight Sensitivity

The engine's behavior is governed by hand-tuned weights, so we analyze how sensitive the outputs are to the two most consequential parameters: the intent-continuity weight w_{int} and the softmax temperature T.

Intent-continuity weight. Holding all other parameters at their defaults (credential intent, $E = 0.05$, $K = 0.145$, target as in Section 9), Table 11 shows $P(\texttt{credential})$ and the sharpness S as w_{int} varies. The action distribution is highly sensitive: raising w_{int} from 0 to 4.5 moves $P(\texttt{credential})$ from 0.255 to 0.969, and sharpness from 0.07 to 0.90. The default 3.0 sits in the steepest part of the curve, meaning small re-tunings materially change how quickly the controller switches to engage: with $w_{\text{int}} = 1.5$ the sharpness just crosses the threshold ($S = 0.302$), while with $w_{\text{int}} = 0$ the distribution stays flat enough that the loop remains in collect (intent is known, so the fallback does not apply).

w_{int}	0.0	1.5	3.0 (default)	4.5
$P(\texttt{credential})$	0.255	0.605	0.873	0.969
sharpness S	0.07	0.30	0.68	0.90
strategy	collect	engage	engage	engage

Table 11. Sensitivity of the predicted distribution and sharpness to the intent-continuity weight w_{int} (credential intent, target as in Section 9); values verified against the reference implementation.

Temperature. The softmax temperature T controls distribution sharpness directly. Table 12 varies T at round 2 of the trace (recon intent, scores $\{4.00, 2.10, 2.00, 1.80, 1.84, 0.50\}$). At $T = 0.5$ the distribution is near-deterministic ($P(\texttt{recon}) = 0.937$, $S = 0.82$); at $T = 2$ it flattens enough that sharpness falls below the threshold and the strategy reverts to collect even though the attacker's intent is already established. Temperature and the sharpness threshold θ must therefore be treated as a joint calibration pair, not independently.

T	0.5	1.0 (default)	2.0
$P(\texttt{recon})$	0.937	0.649	0.384
H (entropy)	0.324	1.181	1.637
sharpness S	0.82	0.34	0.086
strategy	engage	engage	collect

Table 12. Sensitivity of distribution sharpness and strategy to softmax temperature T (round 2 of the trace in Section 9, recon intent, scores $\{4.00, 2.10, 2.00, 1.80, 1.84, 0.50\}$).

Implication for scientific value. These sensitivities are exactly why the rule-based predictors are positioned as *first approximations* rather than calibrated models (Section 5): the current weights encode plausible heuristics, but the outputs are scale-sensitive, and any deployment must either calibrate the weights against real attacker data (Phase 2, Section 11) or replace the scorer with a calibrated model at Seam 2 (Section 6.3). The formal specification (Section 4.3.1–4.5.2) exists precisely so that such calibration and replacement can be done and evaluated without reverse-engineering the implementation.

Relationship to richer models. The loop architecture is model-agnostic: the rule-based state engine and predictor can be replaced by learned models or an I-POMDP^2 solver without changing the graph, engagement, or controller layers. This modularity is a deliberate design property.

The AI convergence question. If Q7 is answered affirmatively, defenders face both a threat and an opportunity: convergent strategies are easier to predict and to guide; diverse strategies (Q8/Q9) resist convergence and require broader engagement portfolios.

13. Conclusion

We presented the Attacker Cognitive Loop, an interpretable rule-based engine for guided active defense that estimates attacker state, predicts action distributions, and selects utility-driven deceptive interventions within a closed loop that switches between containment, information gathering, and active engagement by decision sharpness. We gave the complete formal specification of the scoring, normalization, sharpness, and utility functions (Section 4.3.1–4.5.2), a reproducible four-round closed-loop trace with exact values (Section 9), a sensitivity analysis of the hand-tuned weights (Section 12.1), and simulated per-round latency benchmarks (Section 9.1). The engine is not a paper-only design: it is fully implemented as a modular, unit-tested extension of the open-source ASSCOR platform, with every component located in the public repository (Section 7, Table 4). We stated ten open research questions with evidence ratings and a validation plan, and we were explicit about the boundary between implementation and unvalidated assumption. The primary contribution of this work is the interface contract and reproducible simulation harness. We do not claim defensive efficacy; instead, we offer the community a transparent, modular testbed for falsifying or validating the stated research questions.

The fundamental problem ACL addresses is not the inadequacy of any particular defensive control; it is the structural information asymmetry between attacker and defender. By closing the loop between observation, prediction, and intervention, ACL provides a systematic mechanism to reverse that asymmetry — not by making the defender omniscient, but by making the attacker's uncertainty costly to maintain. We release ACL as a modular, reproducible baseline, and invite the community to replace its heuristic components with more principled models. The ultimate measure of success is not the accuracy of any single prediction, but the defender's ability to act before the attacker's information advantage becomes decisive.

Because the three engine seams (Section 6.3) are narrow interfaces, the community can evaluate better models against this open-source, reproducible baseline. Contact: `chins-xing@proton.me`.

Ethics Statement

This work is exclusively intended for **legitimate defensive purposes**. All capabilities described — attacker state estimation, action prediction, and deceptive engagement — are designed to be deployed only by authorized defenders against adversarial activity within systems and networks they are legally permitted to protect. The authors do not provide or endorse any offensive capability, and we assume all research, experimentation, and validation is conducted in authorized environments in compliance with applicable laws, regulations, and institutional policies. The reference implementation is distributed solely as a defensive security research artifact.

Author Contributions: Conceptualization, J.L.; methodology, J.L.; software, J.L.; validation, J.L.; formal analysis, J.L.; investigation, J.L.; resources, J.L.; data curation, J.L.; writing—original draft preparation, J.L.; writing—review and editing, J.L.; visualization, J.L.; supervision, J.L.; project administration, J.L.; funding acquisition, J.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable. This study did not involve humans or animals; it concerns defensive security software design and simulation.

Informed Consent Statement: Not applicable.

Data Availability Statement: The reference implementation is publicly available at https://github.com/chins-xing/Asscor (branch `ASSCOR-Research-Core`, Apache-2.0). The trace and sensitivity numbers in this paper are regenerated by `cmd/tracecheck` (build tag `tracecheck`) against the reference implementation. The Containerlab

topology (`lunwen/clab-lab/asscor.clab.yml`), the lab scripts and the recorded experiment data are tracked in the repository; the MITRE Caldera installation and the Atomic Red Team source archive are provisioned in the lab environment (Caldera under `/opt/caldera`, Atomic Red Team as a local asset in `lunwen/`).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ACL	Attacker Cognitive Loop
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
AV	Attribution value
DP	Detection probability
IG	Information gain
I-POMDP	Interactive partially observable Markov decision process
MTD	Moving target defense
SRD	Systemic Risk Dynamics
SSAM	System security acceptability model
TTP	Tactics, techniques, and procedures

References

1. Shinde, A.; Doshi, P. Decision-theoretic planning and cognitive modeling for active cyber deception. *Artificial Intelligence* **2026**, *356*, 104540. https://doi.org/10.1016/j.artint.2026.104540.
2. Zhu, Q. Game Theory for Cyber Deception: A Tutorial. In *Proceedings of the 2019 Hot Topics in the Science of Security Symposium and Bootcamp (HoTSoS '19)*, Nashville, TN, USA, 2–3 April 2019; ACM: New York, NY, USA, 2019. https://arxiv.org/abs/1903.01442.
3. Zhang, L.; Thing, V.L.L. Three decades of deception techniques in active cyber defense—Retrospect and outlook. *Computers & Security* **2021**, *106*, 102288. https://doi.org/10.1016/j.cose.2021.102288.
4. Lei, C.; Zhang, H.-Q.; Tan, J.-L.; Zhang, Y.-C.; Liu, X.-H. Moving target defense techniques: A survey. *Security and Communication Networks* **2018**, *2018*, 3759626. https://doi.org/10.1155/2018/3759626.
5. Anwar, A.H.; Kamhoua, C. Game theory on attack graph for cyber deception. In *Decision and Game Theory for Security (GameSec 2020)*; Springer: Cham, Switzerland, 2020. https://doi.org/10.1007/978-3-030-64793-3_24.
6. Caltagirone, S.; Pendergast, A.; Betz, C. *The Diamond Model of Intrusion Analysis*; Technical Report; Center for Cyber Intelligence Analysis and Threat Research (CCIATR): Arlington, VA, USA, 2013.
7. MITRE ATT&CK. Available online: https://attack.mitre.org (accessed on 2026).
8. Verizon. *2026 Data Breach Investigations Report*; Verizon: New York, NY, USA, 2026. Available online: https://www.verizon.com/business/resources/reports/dbir/ (accessed on 2026).
9. ASSCOR Project. *Active Defense Design Whitepaper*, v0.3.0 draft; Internal document: attacker model (Q1–Q10), automated attack/defense chain, deception taxonomy and MTD survey summaries; reference implementation of the ACL engine in `internal/attackerstate`, `internal/predictor`, `internal/engagement`, `internal/defensecycle`, 2026. Repository: https://github.com/chins-xing/Asscor (branch `ASSCOR-Research-Core`, Go 1.26, Apache-2.0).
10. Applebaum, A.; Gaber, D.; Koppen, M.; Polon, T.; Nickels, R.; Lin, J. MITRE Caldera: A scalable, automated adversary emulation platform. Available online: https://github.com/mitre/caldera (accessed on 2026). See also the underlying planning-and-acting-with-unknowns approach of the MITRE adversary-emulation research program, which motivates the planner-driven profile execution used in this paper.
11. Red Canary. Atomic Red Team: Small and Highly Portable Detection Tests Based on MITRE ATT&CK. Available online: https://github.com/redcanaryco/atomic-red-team (accessed on 2026).
12. Shinde, A.; Doshi, P. Modeling behavioral preferences of cyber adversaries using inverse reinforcement learning. *arXiv* **2025**, arXiv:2505.03817.
13. Skopik, F.; et al. Forewarned is forearmed: A survey on large language model-based agents in autonomous cyberattacks. *arXiv* **2025**, arXiv:2505.12786.
14. Kulathumani, V.; Ananthanarayanan, S. Siren: Advancing cybersecurity through deception and adaptive analysis. *arXiv* **2024**, arXiv:2406.06225.

15. Landauer, M.; Skopik, F.; Wurzenberger, M.; Kern, M.; Winter, D. Red team redemption: A structured comparison of open-source tools for adversary emulation. *arXiv* **2024**, arXiv:2408.15645.
16. Bruskin, D.; Zilberman, P. SoK: A survey of open-source threat emulators. *arXiv* **2020**, arXiv:2003.01518.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.